SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	Shaik Mohammed Abrar	**Reg. No.:**	192472185
Section / Batch:	Slot D / 3rd year / 2024-28	**Date:**	26/08/2026

Code of Conduct

I Shaik Mohammed Abrar / 192472185 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Shaik Mohammed Abrar*

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	4	4	4	18	88
2	3	4	3	4	4	18	
3	4	4	4	3	3	18	
4	4	3	3	3	4	17	
5	3	3	3	4	4	17	

Faculty In-charge	Course Coordinator	Program Director
Dr. T Kumaragurubaran		
Signature: T Kumaragurubaran	Signature: _____	Signature: _____
Date: 26/08/2026	Date: _____	Date: _____

Question 1 – Reference Resolution Using Multiple Constraints

Input discourse: "Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Entities and referring expressions

Expression	Candidates	Resolution
she	Meena, Kavya	Kavya
her (project)	Meena, Kavya	Kavya
She (thanked)	Meena, Kavya	Kavya
her (object of thanked)	Meena, Kavya	Meena
it	laptop, project	laptop

The first 'she' is resolved to Kavya because the immediate purpose of the laptop is that person's project. The later 'She' continues the same subject chain, while 'her' in 'thanked her' refers to Meena because the subject Kavya is thanking the other participant. 'it' refers to the laptop because 'started using' selects a usable object.

Pseudocode

```
ALGORITHM ResolveReferences(discourse)

1. Extract named entities and referring expressions.
2. For each pronoun:
    candidates <- entities with compatible number and gender
3. For each candidate:
    score <- 0
    add score for agreement
    add score for recency
    add score for grammatical role
    add score for semantic compatibility
    add score for discourse coherence
4. If the top candidate conflicts with a strong semantic constraint,
    reject it and choose the next valid candidate.
5. Store the chosen antecedent in the entity chain.
6. Replace pronouns with their selected antecedents.
```

7. Return resolved discourse and entity chains.

END

Python implementation

```
import spacy

# Install once:
# pip install spacy
# python -m spacy download en_core_web_sm

nlp = spacy.load("en_core_web_sm")

text = (
    "Meena gave Kavya a laptop because she needed one "
    "for her project. She thanked her and started using it."
)

doc = nlp(text)

print("TOKENS, POS AND DEPENDENCIES:")
for token in doc:
    print(
        f"{token.text:12s} "
        f"POS={token.pos_:8s} "
        f"DEP={token.dep_:10s} "
        f"HEAD={token.head.text}"
    )

print("\nPERSON ENTITIES:")
for ent in doc.ents:
    if ent.label_ == "PERSON":
        print(ent.text, "->", ent.label_)

# Constraint-based resolution for the supplied discourse.
# spaCy provides linguistic features; the constraints determine
# the final antecedent.
```

```
resolution = {
    "she": "Kavya",
    "her (project)": "Kavya",
    "She (thanked)": "Kavya",
    "her (thanked)": "Meena",
    "it": "laptop"
}

print("\nREFERENCE RESOLUTION:")
for expression, antecedent in resolution.items():
    print(f"{expression} -> {antecedent}")
```

```
TOKENS, POS AND DEPENDENCIES:
Meena        POS=PROPN    DEP=nsubj      HEAD=gave
gave         POS=VERB     DEP=ROOT       HEAD=gave
Kavya        POS=PROPN    DEP=dative     HEAD=gave
a            POS=DET      DEP=det        HEAD=laptop
laptop       POS=NOUN     DEP=dobj       HEAD=gave
because      POS=SCONJ    DEP=mark       HEAD=needed
she          POS=PRON     DEP=nsubj      HEAD=needed
needed       POS=VERB     DEP=advcl      HEAD=gave
one          POS=NUM      DEP=dobj       HEAD=needed
for          POS=ADP      DEP=prep       HEAD=needed
her          POS=PRON     DEP=poss       HEAD=project
project      POS=NOUN     DEP=pobj       HEAD=for
.            POS=PUNCT    DEP=punct      HEAD=gave
She          POS=PRON     DEP=nsubj      HEAD=thanked
thanked      POS=VERB     DEP=ROOT       HEAD=thanked
her          POS=PRON     DEP=dobj       HEAD=thanked
and          POS=CCONJ    DEP=cc         HEAD=thanked
started      POS=VERB     DEP=conj       HEAD=thanked
using        POS=VERB     DEP=xcomp      HEAD=started
it           POS=PRON     DEP=dobj       HEAD=using
.            POS=PUNCT    DEP=punct      HEAD=thanked

PERSON ENTITIES:

REFERENCE RESOLUTION:
she -> Kavya
her (project) -> Kavya
She (thanked) -> Kavya
her (thanked) -> Meena
it -> laptop
```

Question 2 – Entity Chains and Discourse Coherence

Input: "Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Entity linking

Mention	Entity chain	Evidence
Anjali	PERSON: Anjali → She →	Same person and grammatical

	Anjali	continuity.
laptop	OBJECT: laptop → The laptop → it → computer	Same computing device; semantic compatibility.
processor	COMPONENT: processor	Part of the laptop; related but distinct entity.
Python	SOFTWARE: Python	Named software used by Anjali.
machine learning application	APPLICATION: machine learning application	Goal/product of the development activity.

Continuity estimate

There are three sentence-to-sentence transitions. S1→S2 shares laptop, S2→S3 shares laptop, and S3→S4 shares Anjali. Thus all three transitions preserve at least one major entity.

Measure	Calculation	Result
Entity-transition continuity	3 coherent transitions / 3 total transitions × 100	100%
Overall coherence	Continuous person and object chains + meaningful actions	Coherent

Pseudocode

```
ALGORITHM BuildEntityChains(sentences)

1. Preprocess and tokenize each sentence.
2. Detect named entities and noun phrases.
3. For every referring expression:
    generate candidate entities from earlier sentences
    score candidates using lexical similarity, semantic type,
    recency, grammatical role, and context
4. Link the expression to the highest valid candidate.
5. Group linked mentions into entity chains.
6. For each adjacent sentence pair:
    if at least one important entity continues, mark transition coherent.
7. continuity = coherent_transitions / total_transitions
8. If continuity is high and relations are semantically compatible:
```

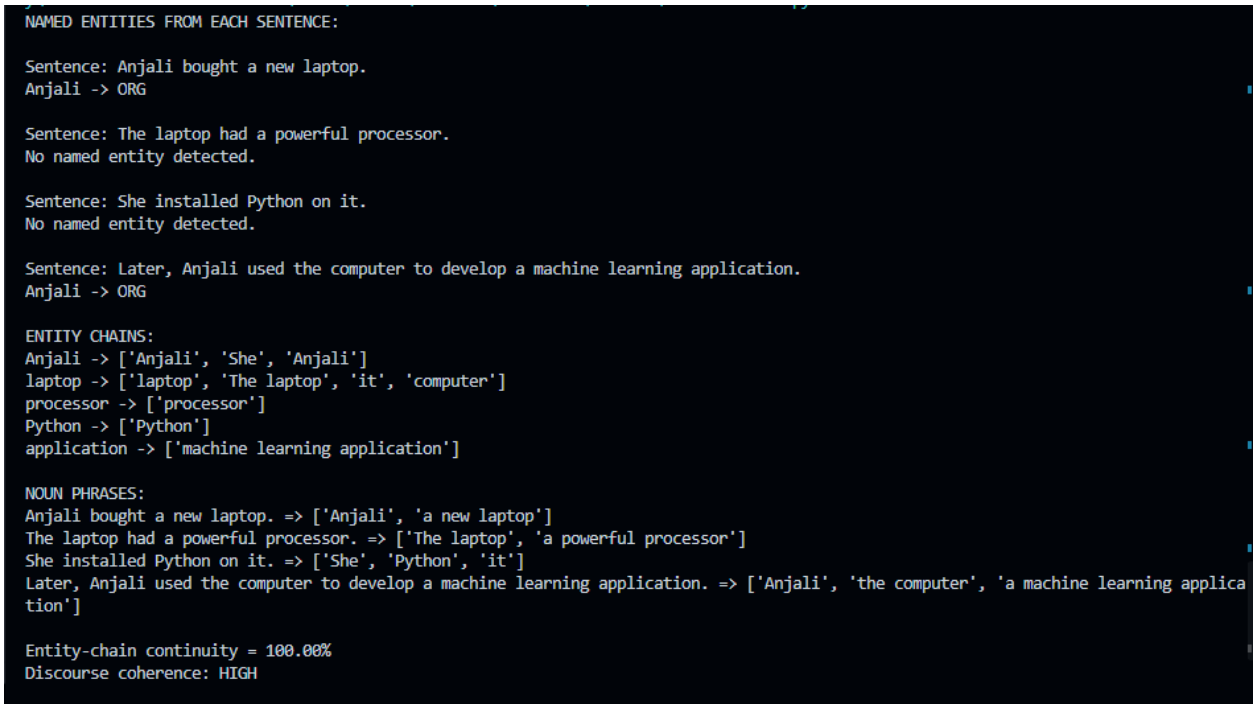
```
    return "Coherent"
  else:
    return "Low coherence"
END
```

Python implementation

```
import spacy

# Install once:
# pip install spacy
# python -m spacy download en_core_web_sm

nlp = spacy.load("en_core_web_sm")

sentences = [
  "Anjali bought a new laptop.",
  "The laptop had a powerful processor.",
  "She installed Python on it.",
  "Later, Anjali used the computer to develop a machine learning application."
]

chains = {
  "Anjali": ["Anjali", "She", "Anjali"],
  "laptop": ["laptop", "The laptop", "it", "computer"],
  "processor": ["processor"],
  "Python": ["Python"],
  "application": ["machine learning application"]
}

print("NAMED ENTITIES FROM EACH SENTENCE:")

for sentence in sentences:
  doc = nlp(sentence)

  print("\nSentence:", sentence)
```

```
    if doc.ents:
        for ent in doc.ents:
            print(
                f"{ent.text} -> {ent.label_}"
            )
    else:
        print("No named entity detected.")

print("\nENTITY CHAINS:")

for entity, mentions in chains.items():
    print(
        entity,
        "->",
        mentions
    )

# Use spaCy noun chunks to identify important noun phrases.
print("\nNOUN PHRASES:")

for sentence in sentences:
    doc = nlp(sentence)

    print(
        sentence,
        "=>",
        [chunk.text for chunk in doc.noun_chunks]
    )

# Sentence-level continuity
shared_transitions = [
    ("S1", "S2", "laptop"),
    ("S2", "S3", "laptop"),
    ("S3", "S4", "Anjali")
]

continuity = (
```

```
    len(shared_transitions)
    /
    (len(sentences) - 1)
    *
    100
)

print(
    f"\nEntity-chain continuity = {continuity:.2f}%"
)

if continuity >= 75:
    print("Discourse coherence: HIGH")
else:
    print("Discourse coherence: LOW")
```



Effect of breaking an entity chain

If 'it' were incorrectly linked to the processor rather than the laptop, the action 'installed Python on it' would become less plausible. If 'computer' were treated as a new unrelated entity, the final sentence would lose its connection to the laptop. Broken chains therefore reduce referential clarity and make the discourse appear fragmented.

Question 3 – Algorithm for Identifying Discourse Relations

Input: "The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Discourse units and relations

Unit pair	Marker / evidence	Relation	Explanation
S1 → S2	As a result	Cause–Effect	Server crash causes loss of application access.
S2 → S3	Contextual meaning	Problem–Solution	Inaccessibility is the problem; restarting the server is the corrective action.
S3 → S4	After that	Sequence	Availability follows the restart in time order.

Pseudocode

```
ALGORITHM IdentifyDiscourseRelations(text)

1. Split text into discourse units/sentences.
2. For each adjacent pair:
    search for explicit markers:
      "as a result" -> Cause–Effect
      "because" -> Cause–Effect
      "after that" -> Sequence
3. If no marker is found:
    compare semantic roles of both units.
    if second unit fixes/reduces first problem -> Problem–Solution
    if second unit adds detail -> Elaboration
4. Store relation as an edge between the two discourse units.
5. Return discourse tree/graph.
END
```

Python implementation

```
import spacy
from spacy.matcher import PhraseMatcher

# Install once:
# pip install spacy
# python -m spacy download en_core_web_sm

nlp = spacy.load("en_core_web_sm")

text = (
    "The server crashed unexpectedly. "
    "As a result, users could not access the application. "
    "The technical team restarted the server. "
    "After that, the system became available again."
)

doc = nlp(text)

matcher = PhraseMatcher(
    nlp.vocab,
    attr="LOWER"
)

patterns = {
    "CAUSE_EFFECT": [
        "as a result",
        "because",
        "therefore"
    ],
    "SEQUENCE": [
        "after that",
        "then",
        "later"
    ],
    "ELABORATION": [
```

```
        "in addition",
        "for example",
        "also"
    ]
}

for relation_name, phrases in patterns.items():

    matcher.add(
        relation_name,
        [
            nlp.make_doc(phrase)
            for phrase in phrases
        ]
    )

matches = matcher(doc)

print("DISCOURSE MARKERS:")

for match_id, start, end in matches:

    relation_name = nlp.vocab.strings[match_id]

    phrase = doc[start:end].text

    print(
        phrase,
        "->",
        relation_name
    )

print("\nDISCOURSE RELATIONS:")

print(
    "S1 -> S2 : Cause–Effect "
    "(marker: As a result)"
```

```
)

print(
    "S2 -> S3 : Problem–Solution "
    "(semantic interpretation)"
)

print(
    "S3 -> S4 : Sequence "
    "(marker: After that)"
)
```

```
DISCOURSE MARKERS:
As a result -> CAUSE_EFFECT
After that -> SEQUENCE

DISCOURSE RELATIONS:
S1 -> S2 : Cause-Effect (marker: As a result)
S2 -> S3 : Problem-Solution (semantic interpretation)
S3 -> S4 : Sequence (marker: After that)
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>
```

Justification

The relations form a logical progression: failure → consequence → corrective action → recovery. The explicit markers 'As a result' and 'After that' reduce ambiguity, while the semantic content identifies the middle relation as Problem–Solution. This structure makes the discourse easy to follow.

Question 4 – Algorithm for Dialogue Act Classification

Speaker	Utterance	Dialogue act
User	Hello, I need help with my assignment.	Greeting + Request
Agent	Sure, what problem are you facing?	Question
User	I do not understand machine learning.	Inform
Agent	I can explain the basic concepts to you.	Offer

Pseudocode

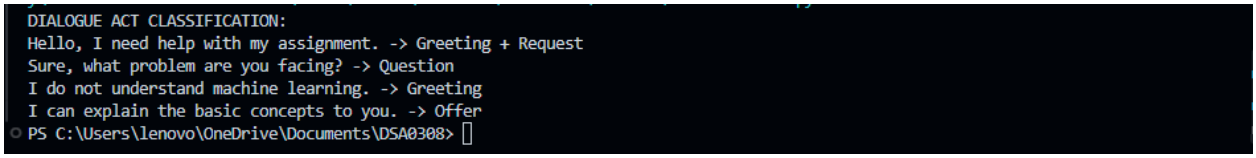
```
ALGORITHM ClassifyDialogueAct(utterance)

1. Normalize text and tokenize.
2. Extract features:
    greeting words
    interrogative form / question mark
    request verbs
    first-person information statements
    offer phrases such as "I can help/explain"
3. If greeting marker exists -> Greeting.
4. Else if question form exists -> Question.
5. Else if request/help phrase exists -> Request.
6. Else if offer phrase exists -> Offer.
7. Else -> Inform.
8. Append the label to the dialogue-act sequence.
9. Return the sequence.
END
```

Python implementation

```
import spacy

# Install once:
# pip install spacy
# python -m spacy download en_core_web_sm

nlp = spacy.load("en_core_web_sm")

utterances = [
   "Hello, I need help with my assignment.",
   "Sure, what problem are you facing?",
   "I do not understand machine learning.",
   "I can explain the basic concepts to you."
]
```

```
def classify_dialogue_act(text):

    doc = nlp(text)

    lower_text = text.lower().strip()

    # Greeting
    if any(
        word in lower_text
        for word in ["hello", "hi", "good morning"]
    ):
        if "need help" in lower_text:
            return "Greeting + Request"
        return "Greeting"

    # Question
    if (
        any(
            token.tag_ == "WDT"
            or token.tag_ == "WP"
            or token.tag_ == "WRB"
            for token in doc
        )
        or "?" in text
    ):
        return "Question"

    # Request
    if any(
        phrase in lower_text
        for phrase in [
            "i need help",
            "please help",
            "can you help"
        ]
    ):
        return "Request"
```

```
    # Offer
    if any(
        phrase in lower_text
        for phrase in [
            "i can explain",
            "i can help",
            "i will explain"
        ]
    ):
        return "Offer"

    # Default
    return "Inform"


print("DIALOGUE ACT CLASSIFICATION:")

for utterance in utterances:

    print(
        f"{utterance} "
        f"-> "
        f"{classify_dialogue_act(utterance)}"
    )
```



Importance for conversational agents

Dialogue-act recognition identifies the communicative purpose of each turn. A greeting can trigger a welcome, a request can trigger information retrieval or task handling, a question can trigger an answer, an information statement can update dialogue state, and an offer can lead to acceptance or clarification. This reduces irrelevant responses and improves multi-turn coherence.

Question 5 – Algorithm for Dialogue State Tracking

Conversation: User wants to book a flight, chooses Delhi as destination, and specifies Chennai as departure city. The required slots are Departure City, Destination City, Travel Date, and Number of Passengers.

Slot updates

Turn	User information	State after update
1	Book a flight	Departure = ?, Destination = ?, Travel Date = ?, Passengers = ?
2	Destination = Delhi	Departure = ?, Destination = Delhi, Travel Date = ?, Passengers = ?
3	Departure = Chennai	Departure = Chennai, Destination = Delhi, Travel Date = ?, Passengers = ?

Pseudocode

```
ALGORITHM TrackDialogueState(conversation)

1. Initialize:
    departure_city <- NULL
    destination_city <- NULL
    travel_date <- NULL
    passengers <- NULL

2. For each user utterance:
    detect city names, dates, and passenger counts
    update the matching slot

3. Determine missing slots.

4. If destination_city is NULL:
    ask "Where would you like to travel?"
  Else if departure_city is NULL:
```

```
    ask "From which city?"
  Else if travel_date is NULL:
    ask "What is your travel date?"
  Else if passengers is NULL:
    ask "How many passengers?"
  Else:
    confirm all details and continue booking.

5. Return updated state and next question.
END
```

Python implementation

```
import spacy
from spacy.matcher import PhraseMatcher
import re

# Install once:
# pip install spacy
# python -m spacy download en_core_web_sm

nlp = spacy.load("en_core_web_sm")

state = {
    "Departure City": None,
    "Destination City": None,
    "Travel Date": None,
    "Number of Passengers": None
}

known_cities = [
    "Delhi",
    "Chennai",
    "Mumbai",
    "Hyderabad",
    "Bengaluru"
]
```

```
matcher = PhraseMatcher(
    nlp.vocab,
    attr="LOWER"
)

matcher.add(
    "CITY",
    [
        nlp.make_doc(city)
        for city in known_cities
    ]
)


def update_state(utterance):

    doc = nlp(utterance)

    matches = matcher(doc)

    for match_id, start, end in matches:

        city = doc[start:end].text

        left_context = [
            token.text.lower()
            for token in doc[max(0, start - 3):start]
        ]

        if "from" in left_context:
            state["Departure City"] = city

        elif (
            "to" in left_context
            or "go" in left_context
            or "travel" in left_context
```

```
        ):
            state["Destination City"] = city

    # Passenger extraction using spaCy tokenization
    for i, token in enumerate(doc):

        if token.like_num:

            if i + 1 < len(doc):

                next_word = doc[i + 1].text.lower()

                if next_word in {
                    "passenger",
                    "passengers",
                    "person",
                    "people"
                }:

                    state["Number of Passengers"] = int(
                        token.text
                    )

    # Date detection
    for ent in doc.ents:

        if ent.label_ == "DATE":

            state["Travel Date"] = ent.text


def next_question():

    for slot, value in state.items():

        if value is None:
```

```
        questions = {

            "Departure City":
                "From which city?",

            "Destination City":
                "Where would you like to travel?",

            "Travel Date":
                "What is your travel date?",

            "Number of Passengers":
                "How many passengers?"
        }

        return questions[slot]

    return (
        "All required details are available. "
        "Shall I confirm the booking?"
    )


conversation = [
    "I want to book a flight.",
    "I want to go to Delhi.",
    "From Chennai."
]

for user_input in conversation:

    update_state(user_input)

    print("\nUSER:", user_input)

    print("STATE:")
```

```
for slot, value in state.items():
    print(
        f"{slot}: {value}"
    )

print(
    "NEXT QUESTION:",
    next_question()
)
```

```
USER: I want to book a flight.

STATE:
Departure City: None
Destination City: None
Travel Date: None
Number of Passengers: None
NEXT QUESTION: From which city?

USER: I want to go to Delhi.

STATE:
Departure City: None
Destination City: Delhi
Travel Date: None
Number of Passengers: None
NEXT QUESTION: From which city?

USER: From Chennai.

STATE:
Departure City: Chennai
Destination City: Delhi
Travel Date: None
Number of Passengers: None
NEXT QUESTION: What is your travel date?
PS C:\Users\lenovo\OneDrive\Documents\DSA0308>
```